Appendix

Discussion on Remark 3

As established in Remark 3, under specific conditions, the ensemble model achieves a tighter upper bound on generalization risks compared to a universal model, i.e., $\mathrm{Upp}(\mathcal{E}'(\widetilde{f}), \delta/3) < \mathrm{Upp}(\mathcal{E}'(\widehat{f}), \delta)$. In GuiDG, we train expert functions $\widehat{f}_i$ with carefully constructed hypothesis spaces $\mathcal{H}_i$. Here, we demonstrate how our design fulfills the condition required in Remark 3, i.e., $\sum_{i=1}^{d} \pi_i' \sqrt{2d_i}/\sqrt{\pi_i} \leq c(\delta)\sqrt{d_0}$, thereby achieving a tighter upper bound on generalization risks.

The upper bound of ensemble risks $\mathrm{Upp}(\mathcal{E}'(\widetilde{f}), \delta)$ depends on VC-dimensions $\widetilde{d}$ and $d_i, i = 1, \ldots, d$, while that of a universal model depends on d_0, the VC-dimension of the function space mapping the entire source domain to the target space. Denote the number of parameters in network space $\mathcal{H}$ as $n(\mathcal{H})$, we have $d_0 = n(\mathcal{H}) \log\{n(\mathcal{H})\}, \widetilde{d} = n(\widetilde{\mathcal{H}}) \log\{n(\widetilde{\mathcal{H}})\}, d_i = n(\mathcal{H}_i) \log\{n(\mathcal{H}_i)\}$. The original source domain inputs possess high dimensionality (e.g., image inputs). The ensemble module (e.g., CMAttn module), instead, only needs to incorporate several domain-specific outputs. Such modules possess much less tunable parameters compared to the function space that directly maps from the source domain, yielding $n(\widetilde{\mathcal{H}}) \ll n(\mathcal{H})$ and $\widetilde{d} \ll d_0$.

By partitioning the whole source dataset into source subdomains, we can simplify the hypothesis space for each subtask. In GuiDG, we adopt prompt-tuning to learn parameter-efficient domain experts for each sub-task. Assuming $\pi_i > \overline{\epsilon} > 0$, application of the Cauchy inequality yields:

$$\begin{aligned}\sum_{i=1}^{d} \pi_i' \sqrt{2d_i}/\sqrt{\pi_i} &\leq \sqrt{\sum_{i=1}^{d} (\pi_i')^2/\pi_i} \sqrt{\sum_{i=1}^{d} 2d_i} \\ &< \sqrt{\sum_{i=1}^{d} 1/\pi_i} \sqrt{\sum_{i=1}^{d} 2d_i}.\end{aligned}$$

Let $\overline{c}_\pi = \sqrt{2\sum_{i=1}^{d} 1/\pi_i}$. The assumption in Corollary 3.2 holds when:

$$\overline{c}_\pi \sqrt{\sum_{i=1}^{d} d_i} \leq c(\delta)\sqrt{d_0}.$$

For a given source domain, $\overline{c}_\pi$ is constant and approximates d when π_i values are similar. Furthermore, since we typically consider tasks at a fixed probability level $1 - \delta$, the terms $(1/d_0)\log(1/\delta)$ and $(1/d_i)\log(3/\delta)$ minimally impact $c(\delta)$ for sufficiently large n. We can establish that $c(\delta) > 1 - \widetilde{\epsilon} > 0$. Thus, the condition:

$$\sum_{i=1}^{d} d_i \leq \{c(\delta)/\overline{c}_\pi\}^2 d_0,$$

is readily achievable by learning domain experts for each sub-task.

As a typical example, when $\pi_i = \pi_i'$ and $c(\delta) \approx 1$, the condition simplifies to $2\sum_{i=1}^{d} d_i \leq d_0$. Let $n_i = n(\mathcal{H}_i)$, $2\sum_{i=1}^{d} n_i \leq n(\mathcal{H})$, we have:

$$\begin{aligned}2\sum_{i=1}^{d} d_i = 2\sum_{i=1}^{d} n_i \log n_i &\leq 2\sum_{i=1}^{d} n_i \log n(\mathcal{H}) \\ &\leq n(\mathcal{H}) \log n(\mathcal{H}) = d_0.\end{aligned}$$

We further provide an illustrative toy example to help understand our proposed bounds. We generate synthetic data nonlinearly from $f(x) = \mathrm{sgn}(x)(3|\cos(x)| + x^2/2 + 3)$ with Gaussian noise. Fully-connected layers with structure $1 \to h \to h \to 1$ are then trained to fit the data. Baseline models include $h = h_1$ hidden unit. Our method splits and fits the data with 2 separate experts, each containing $h = 40$ hidden units. Their outputs are aggregated via a network with 3 hidden units. We test $h_1 =$ 60,80,100 with 40 repeats, each with 200 training and 5000 test samples. Table 6 shows baseline risk R_B, our method's risk R_O, excess risks E_B, E_O, their ratio $R = E_B/E_O$, and theoretical ratio bound r. Both R and r grow as h_1 increases, with r growing faster, validating the bound. R_O is consistently lower than R_B while requiring less parameters, indicating the efficacy of GuiDG.

Table 6: Experimental results on the toy example.

h_1	R_B	R_O	E_B	E_O	R	r
60	0.689	0.599	0.246	0.220	1.118	1.120
80	0.670	0.599	0.263	0.220	1.195	1.482
100	0.659	0.599	0.301	0.220	1.368	1.843

Proof of Theorem 1

Before proving Theorem 1, we introduce the following lemma (Vapnik 2013).

Lemma 4. *Assume hypothesis space $\mathcal{H}$ and $\mathcal{H}_i$ have VC-dimension d_0 and d_i respectively. There exists constant $C > 0$, such that for any $\delta \in (0, 1)$ with probability at least $1 - \delta$ following inequality hold:*

$$\sup_{h \in \mathcal{H}} \mathcal{E}(h) - \sum_{i=1}^{d} \pi_i \widehat{\mathcal{E}}_i(f) \leq C\sqrt{\frac{d_0 \log(n) + \log(1/\delta)}{n}},$$

$$\sup_{h \in \mathcal{H}_i} \mathcal{E}_i(h) - \widehat{\mathcal{E}}_i(h) \leq C\sqrt{\frac{d_i \log(n_i) + \log(1/\delta)}{n_i}}.$$

Proof of Theorem 1. Define the risks on each source domain data in Step 2 as $\widetilde{\mathcal{E}}_i(h) = \sum_{j=1}^{\widetilde{n}_i^S} \frac{1}{\widetilde{n}_i^S} \mathcal{L}(\widetilde{y}_j^i, h(\widetilde{x}_j^i))$, and recall the definition that $\widehat{\mathcal{E}}_i(h) = \sum_{j=1}^{n_i^S} \frac{1}{n_i^S} \mathcal{L}(y_j^i, h(x_j^i))$. As $\widetilde{f} = \mathcal{A}(\widehat{f}_1, \ldots, \widehat{f}_d; \widetilde{D}^S)$ is trained on additional dataset $\widetilde{D}^S$, following lemma 4, with probability at least $1 - \delta$ where $\delta \in (0, 1)$, we have

$$\mathcal{E}_i(\widetilde{f}) \leq \widetilde{\mathcal{E}}_i(\widetilde{f}) + C\sqrt{\frac{\widetilde{d}\log(m) + \log(1/\delta)}{m}}.$$

Algorithm $\mathcal{A}$ aggregates $\widetilde{f}$ using $\widetilde{D}^S$ and ensures that for each source data point $(\widetilde{x}_j^i, \widetilde{y}_j^i)$, $\widetilde{f}$ behaves no worse than any $\widehat{f}_i, i = 1, \ldots, d$. Therefore, we can derive

$$\widetilde{\mathcal{E}}_i(\widetilde{f}) \leq \widetilde{\mathcal{E}}_i(\widehat{f}_i)\,.$$

The risk of $\widetilde{f}$ on P' can be divided. That is, with probability at least $1 - d\delta$, we have

$$\begin{aligned}\mathcal{E}'(\widetilde{f}) =& \sum_{i=1}^{d} \pi_i' \mathcal{E}_i(\widetilde{f}) \\ \leq& \sum_{i=1}^{d} \pi_i' \widetilde{\mathcal{E}}_i(\widetilde{f}) + C\sqrt{\frac{\widetilde{d}\log(m) + \log(1/\delta)}{m}} \\ \leq& \sum_{i=1}^{d} \pi_i' \widetilde{\mathcal{E}}_i(\widehat{f}_i) + C\sqrt{\frac{\widetilde{d}\log(m) + \log(1/\delta)}{m}}\,. \end{aligned} \tag{13}$$

Note that $\widehat{f}_i$ is independent of $\widetilde{D}_i^S$. Using hoeffding inequality we can derive

$$\mathbb{P}\left(\widetilde{\mathcal{E}}_i(\widehat{f}_i) - \mathcal{E}_i(\widehat{f}_i) \leq t\right) \geq 1 - \exp\left(-2\widetilde{n}_i^S t^2 / c_L\right)\,.$$

Let $t = \sqrt{\frac{c_L \log(1/\delta)}{2\widetilde{n}_i^S}}$, we have with probability at least $1 - d\delta, \forall i = 1, \ldots, d$

$$\widetilde{\mathcal{E}}_i(\widehat{f}_i) \leq \mathcal{E}_i(\widehat{f}_i) + \sqrt{\frac{c_L \log(1/\delta)}{2\widetilde{n}_i^S}}\,. \tag{14}$$

Use lemma 4 again, with probability at least $1 - d\delta$, we have for $\forall i = 1, \ldots, d$

$$\mathcal{E}_i(\widehat{f}_i) \leq \widehat{\mathcal{E}}_i(\widehat{f}_i) + C\sqrt{\frac{d_i \log(n_i^S) + \log(1/\delta)}{n_i^S}}\,. \tag{15}$$

Combine inequality (13), (14) and (15) together we have

$$\begin{aligned}\mathcal{E}'(\widetilde{f}) \leq& \sum_{i=1}^{d} \pi_i' \widetilde{\mathcal{E}}_i(\widehat{f}_i) + C\sqrt{\frac{\widetilde{d}\log(m) + \log(1/\delta)}{m}} \\ \leq& \sum_{i=1}^{d} \pi_i' \mathcal{E}_i(\widehat{f}_i) + \left(\sum_{i=1}^{d} \pi_i'/\sqrt{\pi_i}\right)\sqrt{\frac{c_L \log(1/\delta)}{2m}} \\ &+ C\sqrt{\frac{\widetilde{d}\log(m) + \log(1/\delta)}{m}} \\ \leq& \sum_{i=1}^{d} \pi_i' \widehat{\mathcal{E}}_i(\widehat{f}_i) + \left(\sum_{i=1}^{d} \pi_i'/\sqrt{\pi_i}\right)\sqrt{\frac{c_L \log(1/\delta)}{2m}} \\ &+ C\sqrt{\frac{\widetilde{d}\log(m) + \log(1/\delta)}{m}} \\ &+ C\sum_{i=1}^{d} \pi_i' \sqrt{\frac{d_i \log(n_i^S) + \log(1/\delta)}{n_i^S}}\,. \end{aligned} \tag{16}$$

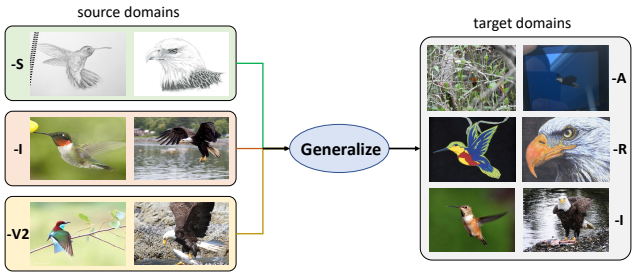

Figure 6: Example pictures of ImageNet-DG. The presented images are from classes 'hummingbird' and 'bald eagle'.

For target risk on $\widehat{f}$, utilize lemma 4 again, with probability at least $1 - d\delta$ we have

$$\mathcal{E}'(\widehat{f}) - \sum_{i=1}^{d} \pi_i' \widehat{\mathcal{E}}_i(\widehat{f}) \leq C\sqrt{\frac{d_0 \log(N) + \log(1/\delta)}{N}}\,. \tag{17}$$

□

Proof of Corollary 2

In this section we prove Corollary 2.

Proof of Corollary 2. As $n_i^S = \pi_i n$, we can reformulate $\mathrm{Upp}(\mathcal{E}'(\widetilde{f}), \delta/3)$ as

$$\begin{aligned}&\mathrm{Upp}(\mathcal{E}'(\widetilde{f}), \delta/3) \\ =& C\sum_{i=1}^{d} \pi_i' \sqrt{\frac{d_i \log(n_i^S) + \log(3/\delta)}{n_i^S}} + \varepsilon \\ =& C\sum_{i=1}^{d} \frac{\pi_i'\sqrt{2d_i}}{\sqrt{\pi_i}} \sqrt{\frac{\log(n_i^S) + (1/d_i)\log(3/\delta)}{N}} + \varepsilon \\ \leq& C\sum_{i=1}^{d} \frac{\pi_i'\sqrt{2d_i}}{c(\delta)\sqrt{\pi_i}} \sqrt{\frac{\log(N) + (1/d_0)\log(1/\delta)}{N}} + \varepsilon \\ \leq& C\sqrt{\frac{d_0 \log(N) + \log(1/\delta)}{N}} + \varepsilon \\ =& \mathrm{Upp}(\mathcal{E}'(\widehat{f}), \delta) + \varepsilon\,, \end{aligned}$$

where the first inequality is due to the definition of $c(\delta)$. □

Details on ImageNet-DG

We construct ImageNet-DG from ImageNet (Deng et al. 2009) and its four variants (ImageNet-A (Hendrycks et al. 2021b), ImageNet-R (Hendrycks et al. 2021a), ImageNet-S (Wang et al. 2019) and ImageNet-V2 (Recht et al. 2019)). ImageNet-DG includes 3 target domains to generalize to, i.e., ImageNet-A, ImageNet (evaluation split) and ImageNet-R. For all 3 tasks, the source domains are ImageNet (train split), ImageNet-S and ImageNet-V2. However, ImageNet-A and ImageNet-R only includes 200 classes sampled from the 1000 classes in ImageNet. In the problem setting of domain generalization, the label distribution $P_{Y|X}$ of source and target domains should be the same. Therefore, we only select source

Table 7: Statistics of ImageNet-DG.

target domain	class count	target samples	sample number of source domains			total
			ImageNet (train)	ImageNet-S	ImageNet-V2	
ImageNet-A	200	7500	259906	10169	2000	272075
ImageNet (eval)	1000	50000	1281167	50889	10000	1342056
ImageNet-R	200	30000	258951	10152	2000	271103

samples that *share categories with the target domain* for each task. The statistics of the resultant ImageNet-DG dataset are in Table 7. Figure 6 presents example pictures in ImageNet-DG. The model needs to incorporate knowledge from various source domains (e.g., sketch and natural style) and generalize to unknown domains. The target domains include natural adversarial (-A) samples that are hard to recognize even for humans, art-style (-R) pictures and natural pictures (-I). The large amount of samples provides robust and comprehensive assessment of model generalization ability.

Detailed Experiment Results

We extend Table 2 in main paper by presenting domain-wise results on TerraIncognita, VLCS and PACS. Results are in Table 8. Each column represents one generalization task. The column names are the target domains. Specifically, in TerraIncognita the columns names are locations of camera, in VLCS (V-VOC2007, L-LabelMe, C-Caltech101, S-SUN09) are names of sub-datasets, and in PACS (P-photo, A-art painting, C-cartoon, S-sketch) are art styles. We can observe that incorporating GuiDG always brings positive overall gains, and that on each dataset our GuiDG achieves the best results. On all tasks, the results are significantly higher than zero-shot CLIP. On TerraIncognita, GuiDG enhances CLIP with abundant domain-specific knowledge, while on VLCS and PACS, GuiDG preserves the pretrained knowledge in CLIP. Therefore, GuiDG achieves consistent superiority on various generalization scenarios.

Implementation Details of GuiDG

Detailed training algorithm for GuiDG are in algorithm 1. Recall the training losses for domain experts as Eq. (18):

$$\mathcal{L}_p^i = -\sum_{j=1}^{n_i^S}\sum_{c=1}^{C}\mathbf{1}[y_j^i = c]\cdot\log P_c(\hat{y}\mid x_j^i, t_{1:C}^i), \tag{18}$$

where $P_c(y|x)$ computes the probability that output y belongs to class c.

During the fine-tuning of CLIP, we first compute domain importance by CMAttn:

$$\mathbf{w}(x) = \mathrm{Softmax}(\cos\langle q(x), [k_1, k_2, \cdots, k_d]\rangle), \tag{19}$$

where $q(\cdot)$ is the query transformation in CMAttn, and k_i are keys transformed from domain experts. We then compute overall loss by weighting the loss for each domain:

$$\mathcal{L}_f = \sum_{i=1}^{d}\sum_{j=1}^{\widetilde{n}_i^S} w_i(x_j^i)\cdot\left(-\sum_{c=1}^{C}\mathbf{1}[y_j^i = c]\cdot\log P_c(\hat{y}|x_j^i, t_{1:C}^i)\right). \tag{20}$$

We implement algorithm 1 and conduct all experiments with PyTorch on one NVIDIA RTX 4090 GPU.

Below we introduce the off-the-shelf regularization techniques $\mathcal{L}_r$ mentioned in Algorithm 1, line 14. In this paper we mainly build our method upon three DG baselines: UEO (Liang et al. 2024), CLIPood (Shu et al. 2023) and WiSE-FT (WS) (Wortsman et al. 2022).

UEO introduces universal entropy minimization as regularization: $\mathcal{L} = \sum_x \tilde{w}(x)\mathcal{H}(p(x)) - \mathcal{H}(\bar{p})$, where $\mathcal{H}(p(x)) = -\sum_{c=1}^{C} p_c(x)\log p_c(x)$ denotes the Shannon entropy of $p(x)$, $\tilde{w}(x) = \frac{1}{B}$ where B is batch size, $p_c(x)$ is classification probability for class c. The regularization weight is set to $\alpha = 0.1$ as in (Liang et al. 2024).

CLIPood introduces beta moving average (BMA) for weight averaging, and margin metric softmax (MMS) loss. BMA maintains a moving average model θ^{BMA} and at each time step t, and the current model θ^t is added into θ_t^{BMA} to update the moving average:

$$\theta_t^{\text{BMA}} = \frac{\sum_{k=0}^{t-1}\alpha_k}{\sum_{k=0}^{t}\alpha_k}\cdot\theta_{t-1}^{\text{BMA}} + \frac{\alpha_t}{\sum_{k=0}^{t}\alpha_k}\cdot\theta_t, \tag{21}$$

where

$$\alpha_t = \mathrm{Beta}(\beta,\beta)\left(\frac{t+0.5}{T+1}\right), \tag{22}$$

where $\mathrm{Beta}(\beta,\beta)$ is beta distribution and $\beta = 0.5$ is hyperparameter. The MMS loss is given by:

$$\mathcal{L} = -\log\frac{\exp(S(I_x, T_y)/\tau)}{\sum_{c=1}^{C}\exp(S(I_x, T_c) + \lambda\cdot D(T_y, T_c))/\tau}, \tag{23}$$

where $D(T_y, T_c) = 1 - S(T_y, T_c)$, $S(\cdot,\cdot)$ is similarity score between feature representations.

WiSE-FT is another weight averaging technique. Consider pretrained model weight β_0 and weights after fine-tuning β_1, the weight-space ensemble is given by:

$$\beta_{ens} = (1-\alpha)\cdot\beta_0 + \alpha\cdot\beta_1, \tag{24}$$

where $\alpha = 0.5$ is hyperparameter.

For more detailed description please refer to the original papers (Liang et al. 2024; Shu et al. 2023; Wortsman et al. 2022). Our method is compatible with more generalization techniques, which could potentially further improve the performance.

Table 8: Detailed results on TerraIncognita, VLCS and PACS. Best results are in bold.

Methods	TerraIncognita					VLCS					PACS				
	L100	L38	L43	L46	Avg.	C	L	S	V	Avg.	A	C	P	S	Avg.
CLIP (Radford et al. 2021)	50.8	23.4	32.2	28.8	33.8	**100.0**	67.4	73.5	86.1	81.8	97.6	98.9	**100.0**	88.2	96.2
ERM (WF)	57.1	54.8	48.5	43.6	51.0	**100.0**	66.1	76.8	90.1	83.3	97.8	99.1	**100.0**	89.9	96.7
ERM (WF)+ GuiDG	59.2	56.0	50.1	46.2	52.9	**100.0**	68.0	77.4	89.9	83.8	98.5	99.4	**100.0**	**92.5**	**97.6**
UEO (WF) (Liang et al. 2024)	58.7	57.5	47.6	42.2	51.5	**100.0**	60.1	76.2	88.9	81.3	98.1	98.9	**100.0**	90.7	96.9
UEO (WF)+ GuiDG	57.7	**60.0**	48.2	43.4	52.3	**100.0**	66.9	**79.6**	90.1	84.2	98.9	**99.6**	**100.0**	91.8	**97.6**
CLIPood (Shu et al. 2023)	73.1	58.4	**57.7**	50.9	60.0	98.9	68.2	78.8	**90.8**	84.2	**99.0**	**99.6**	**100.0**	90.7	97.3
CLIPood+ GuiDG	**74.7**	59.8	56.8	**52.3**	**60.9**	99.3	**69.7**	77.9	90.1	**84.3**	98.3	**99.6**	**100.0**	91.1	97.3

Algorithm 1: Two-step training algorithm for GuiDG.

procedure TRAINING DOMAIN EXPERTS.
  Input: source dataset D^S, number of source domains d, CLIP encoders E_v and E_t.
  for i in $[1, 2, ..., d]$ **do**
    Obtain domain-specific data D_i^S from D^S (which is readily available in multi-source DG tasks, and is randomly divided in single-source DG tasks).
    while not converged **do**
      Sample data points (x_j^i, y_j^i) from D_i^S.
      Compute training loss $\mathcal{L}_p^i$ in Eq. (18).
      Update learnable prompt embeddings $\mathbf{p}_i$ by minimizing $\mathcal{L}_p^i$.
    end while
  end for
  return Trained domain experts $\{\mathbf{p}_i\}_{i=1}^d$.
end procedure
procedure FINE-TUNING CLIP WITH DEDICATED PROMPT GUIDANCE.
  Input: Trained domain experts $\{\mathbf{p}_i\}_{i=1}^d$, additional dataset $\widetilde{D}^S$, off-the-shelf fine-tuning regularization term $\mathcal{L}_r$, regularization weight α, CLIP encoders E_v and E_t.
  while not converged **do**
    Sample data points (x_j, y_j) from $\widetilde{D}^S$.
    Compute domain weights as in Eq. (19).
    Compute training loss $\mathcal{L}_f$ in Eq. (20).
    Update parameters in E_v and CMAttn by minimizing $\mathcal{L}_f$.
  end while
  return Fine-tuned CLIP vision encoder parameters θ_{E_v}, trained CMAttn with parameters $\{\theta_{L_q}, \theta_{L_k}\}$.
end procedure